

COMPTE RENDU TECHNIQUE DE PROJET DE FIN DE MODULE DEEP LEARNING

Conception et Développement d'un Système Intelligent de Détection en Temps Réel du Niveau d'Engagement (Concentration) par Vision par Ordinateur et Deep Learning

Module 1 : Engagement par analyse faciale (webcam)

Module 2 : Engagement par interactions clavier / souris

Réalisé par :

Yahia Ez-zemmouri

Mohamed Amine Tazi

Date :

22/06/2026

Environnement d'entraînement :

JupyterLab dans un conteneur Docker - GPU NVIDIA GeForce RTX 3060 Ti (CUDA)

Google collab pour la souris et clavier

Table des Matières

Table des Matières	1
1. Présentation Générale du Projet.....	1
1.1 Contexte et objectif.....	1
1.2 Approche retenue.....	1
1.3 Vue d'ensemble du pipeline	1
1.4 Vision d'intégration avec la plateforme e-learning	1
2. Arborescence du Projet.....	1
3. Environnement d'Entraînement (Docker + GPU)	1
3.1 Pourquoi un conteneur Docker	1
3.2 Configuration matérielle	1
4. Phase 1 - Extraction des Caractéristiques (Features)	1
4.1 Principe	1
4.2 Résultat de cette phase.....	1
5. Phase 2 - Entraînement du Modèle Bi-LSTM.....	1
5.1 Pourquoi un Bi-LSTM.....	1
5.2 Résultats obtenus	1
6. Phase 3 — Service d'Inférence (API FastAPI)	1
6.1 Objectif de cette phase.....	1
7. Phase 4 - Démonstration en Webcam Live.....	1
7.1 Objectif de cette phase.....	1
7.2 Constat de cette phase.....	1
8. Suivi du Projet.....	1
9. Synthèse et Prochaines Étapes	1
9.1 Ce qui est fonctionnel aujourd'hui	1
9.2 Prochaines étapes prévues	1
II : Clavier et souris	
1. Introduction et Contexte	1
1.1 Problématique	1
1.2 Objectif du projet	1
1.3 Contributions.....	1
2. Première Approche — Datasets Publics Génériques.....	2
2.1 CMU Keystroke Dynamics Dataset.....	2
2.1.1 Structure des colonnes	6
2.2 Balabit Mouse Dynamics Challenge Dataset.....	6
2.2.1 Structure des fichiers.....	7
3. Feature Engineering	8

3.1 Features Clavier (31 features).....	8
3.2 Features Souris (8 features).....	8
3.3 Fenêtres Glissantes et Séquences.....	9
4. Diagnostic Critique — Le Problème du Data Leakage.....	10
4.1 Constat initial.....	10
4.2 Cause identifiée.....	10
4.3 Implication méthodologique.....	10
5. Architecture du Modèle — Late Fusion Bi-LSTM.....	11
5.1 Choix de l’architecture.....	11
5.2 Architecture Bi-LSTM.....	11
5.3 Paramètres d’entraînement.....	11
6. Pipeline de Développement et Déploiement.....	13
6.1 Artefacts de production.....	13
6.2 API FastAPI et format des requêtes.....	13
6.3 Résilience au changement d’onglet.....	15
7. Système de Collecte de Données Réelles.....	16
7.1 Motivation.....	16
7.2 Conception de la page de collecte.....	16
7.3 Architecture Late Fusion présentée pendant la collecte.....	16
7.4 Le problème des labels heuristiques, illustré en conditions réelles.....	17
7.5 Écran d’auto-annotation.....	18
7.6 Sauvegarde et bilan de session.....	18
7.7 Détection de l’inactivité pendant la collecte.....	19
7.8 Format des données collectées.....	19
8. Réentraînement sur Données Réelles — Protocole Anti-Leakage.....	20
8.1 Un second risque de fuite de données.....	20
8.2 Protocole retenu.....	20
8.3 Volume de données collecté.....	20
8.4 Limite reconnue.....	20
9. Résultats et Évaluation.....	21
9.1 Métriques d’évaluation.....	21
9.2 Comparaison des versions du modèle.....	21
9.3 Analyse critique des erreurs.....	21
10. Perspectives et Travaux Futurs.....	22
10.1 Élargissement de la collecte de données.....	22
10.2 Intégration avec la modalité webcam.....	22
10.3 Dashboard enseignant.....	22
10.4 Déploiement Docker et réentraînement continu.....	22
11. Conclusion.....	23

Références.....	24
9. Synthèse et Prochaines Étapes	25
9.1 Ce qui est fonctionnel aujourd'hui	25
9.2 Prochaines étapes prévues.....	25

1. Présentation Générale du Projet

1.1 Contexte et objectif

Avec la généralisation des cours en ligne (e-learning, classes virtuelles), les enseignants perdent l'un des repères les plus naturels du présentiel : la capacité d'observer en direct si les étudiants suivent, décrochent ou sont confus. Une visioconférence classique ne donne aucune indication objective sur l'attention réelle des participants l'enseignant voit des vignettes vidéo, mais pas un niveau d'engagement mesurable.

Ce projet répond à ce besoin en développant un système assisté par l'intelligence artificielle capable de détecter, en temps réel et à partir d'un flux webcam et des mouvements clavier souris de l'étudiant, son niveau d'engagement (concentration) pendant une session de cours en ligne. L'objectif final est d'intégrer ce système comme module additionnel à une plateforme e-learning existante, pour donner aux enseignants une visibilité agrégée sur l'attention de leur audience pendant qu'ils donnent un cours.

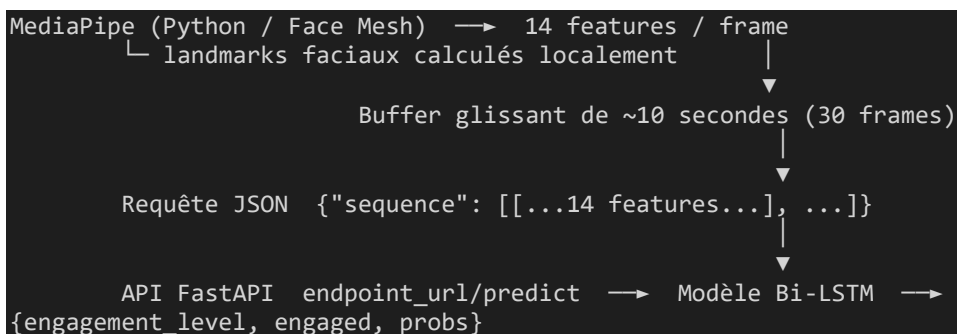
1.2 Approche retenue

Le projet repose sur une approche multimodale prévue en deux volets :

- Module 1 - Engagement par analyse faciale (webcam) : ce module, entièrement développé et fonctionnel, est l'objet de ce compte rendu. Il extrait des caractéristiques du visage de l'étudiant via MediaPipe et classe le niveau d'engagement à l'aide d'un modèle Bi-LSTM entraîné sur le dataset DAiSEE.
- Module 2 - Engagement par interactions clavier/souris (à venir) : ce module complétera l'analyse faciale par des signaux comportementaux (fréquence de frappe, mouvements de souris, inactivité prolongée, scroll, changement d'onglet, etc.), pour fournir un score d'engagement combiné, plus robuste face aux limites de la seule vision par ordinateur (visage hors champ, mauvaise luminosité, webcam coupée).

Le module Le développement du module d'analyse clavier/souris est en cours de finalisation suite à des limitations méthodologiques liées aux données initiales. En effet, l'entraînement préliminaire du modèle à partir des bases de données CMU et Balabit s'est avéré infructueux en raison de leur absence de labellisation. Faute de labels, le modèle n'a pu réaliser un apprentissage supervisé effectif et s'est limité à l'application de transformations mathématiques statiques sur les nouvelles données, provoquant un phénomène de fuite de données (data leakage). Pour y remédier, un protocole de collecte interne a été mis en œuvre via une plateforme développée localement et exposée sur le web grâce à Ngrok. Cette infrastructure permet de capturer et de labelliser précisément la dynamique comportementale des utilisateurs lors de leurs interactions à l'écran, et un extrait (snippet) de ce nouveau jeu de données est présenté ci-après.

1.3 Vue d'ensemble du pipeline



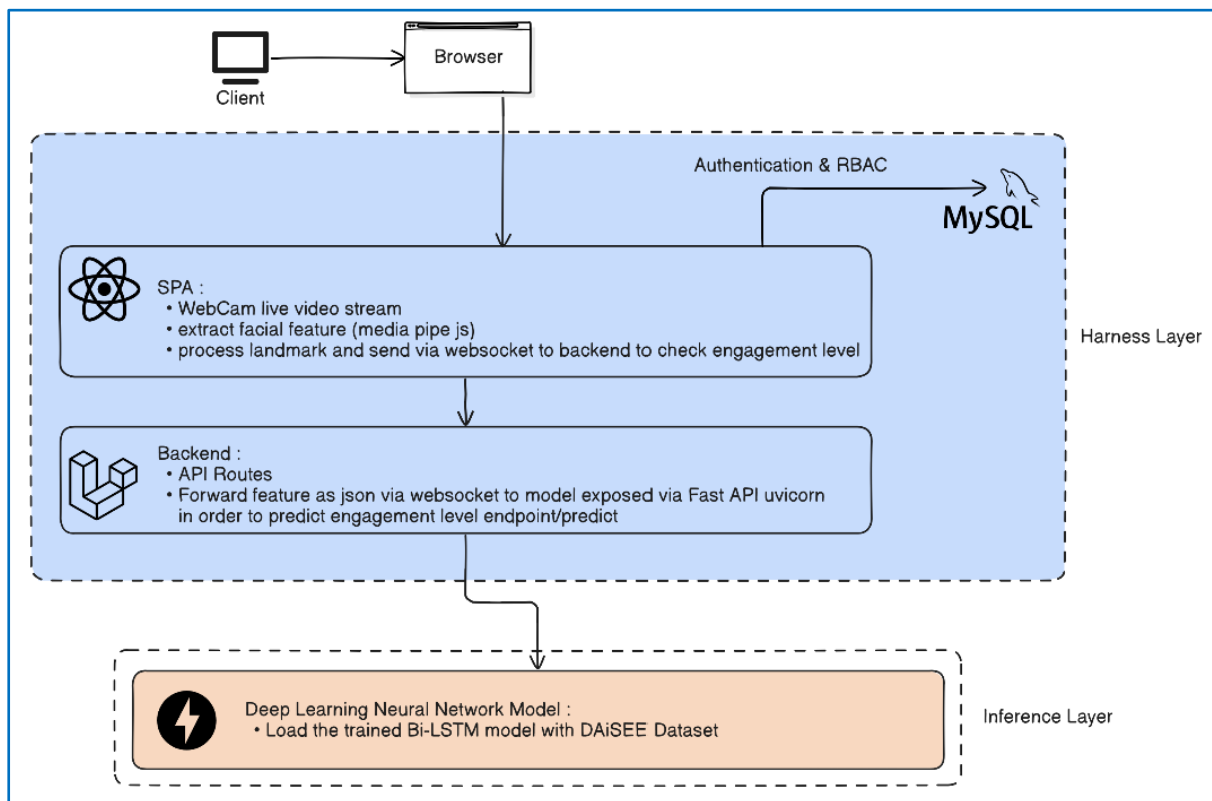
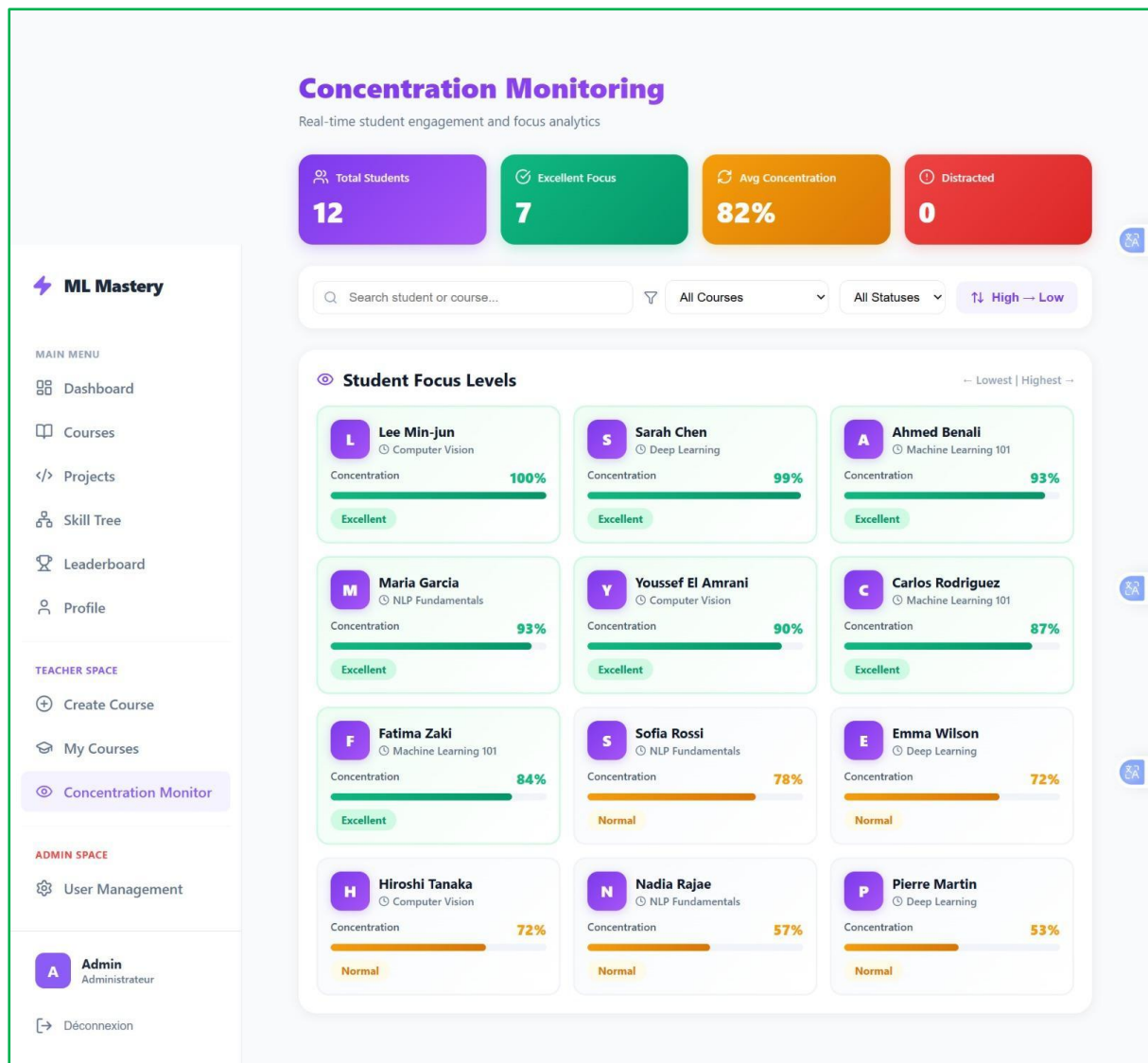


Figure 1 - Schéma global de la solution

1.4 Vision d'intégration avec la plateforme e-learning

L'objectif à terme n'est pas de livrer une application autonome, mais un service d'engagement intégrable à une plateforme e-learning. Le scénario d'usage visé est le suivant :

1. L'étudiant rejoint une session de cours en ligne depuis son navigateur ou son client de visioconférence intégré à la plateforme.
2. Avec son consentement, la webcam de l'étudiant est échantillonnée localement (ou via un petit client léger) pour extraire les features faciales aucune vidéo brute n'a besoin d'être transmise au serveur, seules les caractéristiques numériques (14 valeurs par frame) le sont.
3. Ces séquences sont envoyées en continu à l'API d'inférence (le service FastAPI décrit dans ce document), qui retourne un niveau d'engagement et des probabilités associées toutes les ~10 secondes.
4. La plateforme e-learning agrège ces résultats pour tous les étudiants connectés et les présente à l'enseignant sous forme de tableau de bord en temps réel (ex. : liste d'étudiants avec un indicateur coloré de concentration, alerte si le niveau moyen de la classe chute, historique par étudiant en fin de session).
5. Une fois développé, le Module 2 (clavier/souris) viendra enrichir ce score avec des signaux comportementaux complémentaires, pour des étudiants effectuant des exercices interactifs (quiz, qcm, question de cours).



2. Arborescence du Projet

Le projet d'entraînement et de service du modèle est organisé comme suit (dossier training/) :

```

training/
├── DataSet/                # Vidéos brutes du dataset DAiSEE
├── GenderClips/           # Métadonnées / clips fournis avec DAiSEE
├── Labels/                # Fichiers de labels (engagement, boredom,
...)
├── extractFrames.py       # Script d'origine DAiSEE (extraction de
frames)
├── hog.py                 # Script d'origine DAiSEE (features HOG)
├── engagement/           # >>> Cœur du projet EngageSense <<<
│   ├── 01_extract_features.ipynb # Notebook : extraction des features DAiSEE
│   ├── 02_train_bilstm.ipynb    # Notebook : entraînement du modèle Bi-LSTM
│   ├── daisee_features.py       # Module : extraction de features MediaPipe
│   ├── serve.py                 # API FastAPI de prédiction
│   ├── webcam_live.py          # Script CLI : prédiction en webcam live
│   └── make_test_payload.py     # Génère un payload JSON de test pour test
de l'entraînement via Postman
│   ├── requirements.txt        # Dépendances (entraînement)
│   ├── requirements-serve.txt  # Dépendances (serving uniquement)
│   └── Dockerfile              # Image Docker (JupyterLab + GPU,
python:3.11-slim deployment des artifacts du model et exposition uvicorn fastapi)
└── README.md                  # Documentation du projet

```

La section suivante détaille le rôle de chaque fichier et la manière dont ils s'enchaînent dans le pipeline global : extraction de features → entraînement → service d'inférence → démonstration en temps réel.

3. Environnement d'Entraînement (Docker + GPU)

3.1 Pourquoi un conteneur Docker

L'entraînement du modèle Bi-LSTM nécessite un environnement reproductible avec les bonnes versions de CUDA, TensorFlow/PyTorch et MediaPipe. Plutôt que d'installer ces dépendances directement sur la machine hôte (avec les risques de conflits de versions que cela implique), l'environnement d'entraînement a été packagé dans une image Docker, exécutant une instance JupyterLab accessible depuis le navigateur.

3.2 Configuration matérielle

Composant	Détail
Device	cuda
GPU	NVIDIA GeForce RTX 3060 Ti
Interface de développement	JupyterLab (dans le conteneur)
Hôte	Serveur Rocky Linux

Image Docker

Une image docker a été utiliser qui contient un environnement jupyter lab avec les toolkits requis pour la consommation de la GPU dans la machine : **quay.io/jupyter/pytorch-notebook:cuda12-python-3.11**

```
[root@homelab ~]# docker exec -it 27fcaefbcfa7 /bin/bash
(base) jovyan@27fcaefbcfa7:~$
(base) jovyan@27fcaefbcfa7:~$
(base) jovyan@27fcaefbcfa7:~$ nvidia-smi
Sat Jun 20 14:52:05 2026
+-----+
| NVIDIA-SMI 580.159.03                  Driver Version: 580.159.03          CUDA Version: 13.0               |
+-----+-----+-----+-----+-----+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap       |      0x00000000:01:00.0 On | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
| 0     NVIDIA GeForce RTX 3060 Ti        Off          | 00000000:01:00.0 On |          0%      Default |
| 31%   35C   P8             13W / 200W   |          9MiB / 8192MiB |                      N/A  |
+-----+-----+-----+-----+-----+-----+
+-----+
| Processes:                               |
|  GPU   GI    CI          PID    Type   Process name                  GPU Memory |
|        ID    ID                                   |            Usage   |
+-----+-----+-----+-----+-----+
| No running processes found              |
+-----+
(base) jovyan@27fcaefbcfa7:~$
```

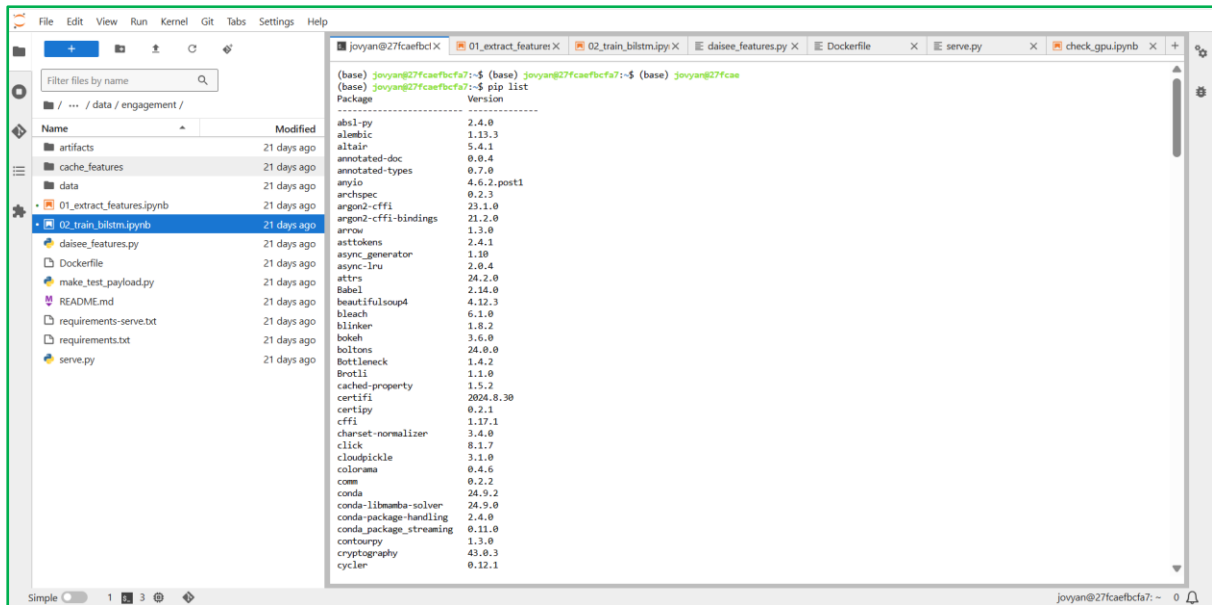


Figure 2 — Conteneur Docker actif avec accès GPU (nvidia-smi / vérification CUDA dans le notebook)

requirements.txt / requirements-serve.txt

Deux fichiers de dépendances distincts ont été créés pour séparer les besoins :

- requirements.txt : dépendances complètes nécessaires à l'entraînement (TensorFlow/Keras avec support GPU, MediaPipe, OpenCV, pandas, numpy, jupyterlab, etc.).
- requirements-serve.txt : sous-ensemble minimal nécessaire uniquement pour exécuter l'API de service (FastAPI, MediaPipe, le runtime du modèle), afin de garder l'environnement de production aussi léger que possible.

4. Phase 1 - Extraction des Caractéristiques (Features)

4.1 Principe

Plutôt que d'entraîner un modèle directement sur des frames vidéo brutes (coûteux en calcul et peu interprétable), le choix a été fait d'extraire des caractéristiques géométriques et comportementales du visage à partir des landmarks détectés par MediaPipe. Chaque frame du dataset DAiSEE est ainsi réduite à un vecteur de 14 features numériques, et chaque vidéo (10 secondes) devient une séquence de 30 vecteurs (à raison d'environ 3 frames par seconde échantillonnées).

Cette approche présente plusieurs avantages pour le cas d'usage e-learning visé : elle réduit drastiquement la quantité de données à transmettre (quelques dizaines de nombres par frame plutôt qu'une image complète), elle s'exécute rapidement même sans GPU côté client, et elle évite de transmettre/stocker des images du visage de l'étudiant un point important pour la confidentialité dans un contexte scolaire.

`daisee_features.py`

Ce module est le cœur de l'extraction de features. Il encapsule la logique MediaPipe (Face Mesh / Face Landmarker) et calcule, pour chaque frame d'une vidéo, le vecteur de 14 caractéristiques utilisées ensuite par le modèle. C'est ce même module qui est réutilisé à l'identique lors de l'entraînement (extraction batch sur DAiSEE) et lors de l'inférence en temps réel (`webcam_live.py`, `serve.py`) - garantissant que les features vues par le modèle en production sont calculées exactement de la même façon qu'à l'entraînement.

Parmi les caractéristiques typiquement extraites à partir des landmarks faciaux (à détailler/ajuster selon votre implémentation exacte dans `daisee_features.py`) :

- Eye Aspect Ratio (EAR) - œil gauche et œil droit : mesure l'ouverture des yeux, utile pour détecter les clignements et la somnolence.
- Mouth Aspect Ratio (MAR) : mesure l'ouverture de la bouche (bâillements, prise de parole).
- Orientation de la tête (yaw / pitch / roll) : estimée à partir des landmarks 3D, indique si l'étudiant regarde l'écran ou détourne la tête.
- Indicateurs liés au regard / sourcils : proxy d'attention ou de confusion.
- Variations inter-frames de certains landmarks : capture la dynamique du mouvement facial sur la fenêtre temporelle.

```

43 FEATURE_NAMES = [
44     "face_present",
45     "ear_left", "ear_right",
46     "mar", "mouth_width",
47     "brow_left", "brow_right",
48     "gaze_left_x", "gaze_left_y",
49     "gaze_right_x", "gaze_right_y",
50     "yaw", "pitch", "roll",
51 ]
52 N_FEATURES = len(FEATURE_NAMES)
53

```

`01_extract_features.ipynb` (+ `01_extract_features.ipynb.extracted.py`)

Ce notebook orchestre l'extraction à grande échelle sur l'ensemble du dataset DAiSEE : il parcourt les vidéos du dossier DataSet/, lit les labels d'engagement associés dans Labels/, appelle `daisee_features.py` sur chaque vidéo pour produire la séquence de 30×14 features, puis sauvegarde le résultat dans un format exploitable pour l'entraînement (typiquement un ou plusieurs fichiers numpy/CSV contenant les séquences et leurs labels).

The image displays two screenshots of a Jupyter Notebook interface, showing the code for feature extraction from the DAiSEE dataset.

Top Screenshot: The notebook is titled "01_extract_features.ipynb". The code cell [1] defines the environment and parameters:

```
[1]: import os, glob, numpy as np, pandas as pd
from pathlib import Path
import daisee_features as F # our shared, parity-safe feature module

ROOT = Path(".") # DAiSEE root: contains DataSet/ and Labels/
DATASET = ROOT / "DAiSEE" / "DataSet"
LABELS = ROOT / "DAiSEE" / "Labels"
CACHE = Path("cache_features") # per-subject cached features
OUTDIR = Path("data") # final X/y .npy files
SEQ_LEN = 30 # frames sampled per 10s clip
TARGET = "Engagement" # label column to predict (0..3)
N_JOBS = 6 # CPU cores for parallel extraction
CACHE.mkdir(exist_ok=True); OUTDIR.mkdir(exist_ok=True)

SPLITS = ("Train": "TrainLabels.csv", "Validation": "ValidationLabels.csv", "Test": "TestLabels.csv")
print("Feature vector:", F.FEATURE_NAMES)
```

The feature vector is listed as: `['face_present', 'ear_left', 'ear_right', 'mar', 'mouth_width', 'brow_left', 'brow_right', 'gaze_left_x', 'gaze_left_y', 'gaze_right_x', 'gaze_right_y', 'yaw', 'pitch', 'roll']`.

Cell [2] defines the `load_labels` and `find_video` functions:

```
[2]: def load_labels(csv):
    df = pd.read_csv(LABELS / csv)
    df.columns = [c.strip() for c in df.columns] # header has trailing spaces
    df["clip"] = df["clipID"].astype(str).strip().str.replace(r"(\.avi|mp4)$", "", regex=True)
    return df.set_index("clip").to_dict() # {clipid_no_ext: label}

def find_video(split, clip):
    # clip folder = clip id (no extension); video file inside is .avi or .mp4
    for subj in glob.glob(str(DATASET/split+"*")):
        d = Path(subj)/clip
        if d.is_dir():
            for ext in ("avi", "mp4"):
                vids = list(d.glob(f"*.{ext}"))
                if vids: return vids[0]
    return None
```

Bottom Screenshot: The notebook is titled "02_train_bilstm.ipynb". The code cell [2] defines the `load_labels` and `find_video` functions (repeated from the top screenshot) and the `index_videos` function:

```
[2]: def load_labels(csv):
    df = pd.read_csv(LABELS / csv)
    df.columns = [c.strip() for c in df.columns] # header has trailing spaces
    df["clip"] = df["clipID"].astype(str).strip().str.replace(r"(\.avi|mp4)$", "", regex=True)
    return df.set_index("clip").to_dict() # {clipid_no_ext: label}

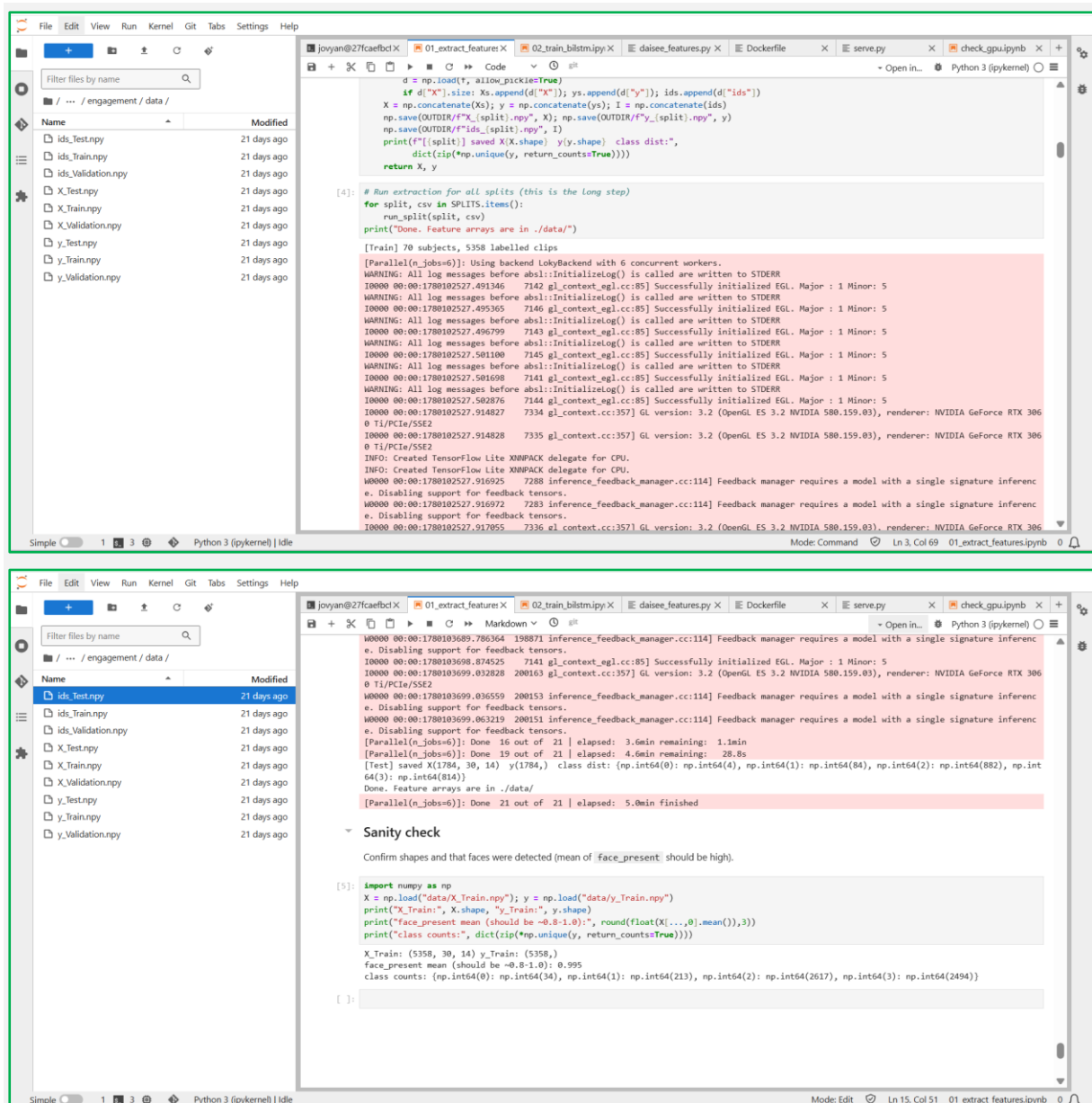
def find_video(split, clip):
    # clip folder = clip id (no extension); video file inside is .avi or .mp4
    for subj in glob.glob(str(DATASET/split+"*")):
        d = Path(subj)/clip
        if d.is_dir():
            for ext in ("avi", "mp4"):
                vids = list(d.glob(f"*.{ext}"))
                if vids: return vids[0]
    return None

# Faster: build a clip -> path index once per split (avoids scanning every time)
def index_videos(split):
    idx = {}
    for vid in glob.glob(str(DATASET/split+"*/*/*")):
        p = Path(vid)
        if p.suffix.lower() in (".avi", ".mp4"):
            idx[p.stem] = p
    return idx
```

Cell [3] defines the `process_subject` function:

```
[3]: from joblib import Parallel, delayed

def process_subject(split, subj_dir, labels):
    """Extract features for all clips of one subject. Cached to disk -> resumable."""
    import mediapipe as mp
    subj = Path(subj_dir).name
    cache_file = CACHE / f"{split}_{subj}.npz"
    if cache_file.exists():
        return str(cache_file)
    fm = mp.solutions.face_mesh.FaceMesh(static_image_mode=False, max_num_faces=1,
                                          refine_landmarks=True, min_detection_confidence=0.5)
```



4.2 Résultat de cette phase

À l'issue de cette étape, le dataset brut (vidéos DAiSEE) est transformé en un ensemble de séquences numériques prêtes pour l'entraînement supervisé : pour chaque clip, une matrice de forme (30, 14) associée à son label d'engagement (Very Low / Low / High / Very High, selon l'annotation DAiSEE).

5. Phase 2 - Entraînement du Modèle Bi-LSTM

5.1 Pourquoi un Bi-LSTM

L'engagement ne se déduit pas d'une image isolée mais de sa dynamique sur quelques secondes (un sourire furtif, un regard qui décroche puis revient, un bâillement). Un réseau récurrent bidirectionnel (Bi-LSTM) est adapté à ce problème : il traite la séquence de 30 pas de temps (features) en tenant compte à la fois du contexte passé et futur de la séquence pour décider du niveau d'engagement global sur la fenêtre de ~10 secondes.

02_train_bilstm.ipynb

Ce notebook, exécuté dans l'environnement JupyterLab/Docker avec accès GPU, réalise l'entraînement complet du modèle :

1. Chargement des séquences de features et des labels produits à la phase précédente.
2. Découpage en ensembles d'entraînement, de validation et de test.
3. Normalisation des features et encodage des classes d'engagement.
4. Construction de l'architecture Bi-LSTM (couche(s) Bidirectional LSTM, Dropout, couche(s) Dense, sortie softmax pour les niveaux d'engagement).
5. Entraînement sur GPU (device cuda — RTX 3060 Ti), avec suivi des courbes de loss/accuracy.
6. Évaluation sur l'ensemble de test et sauvegarde du modèle entraîné (poids + architecture) pour réutilisation par l'API de service.

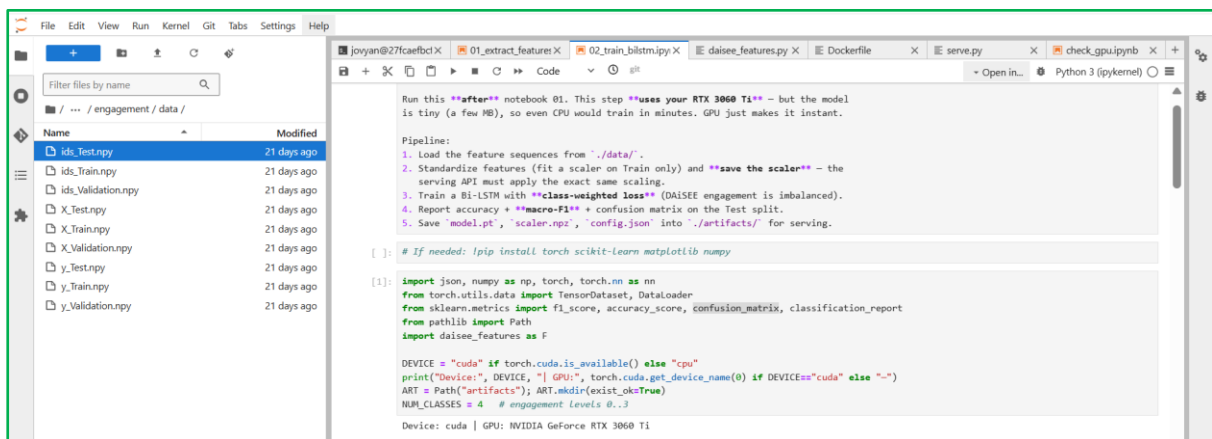
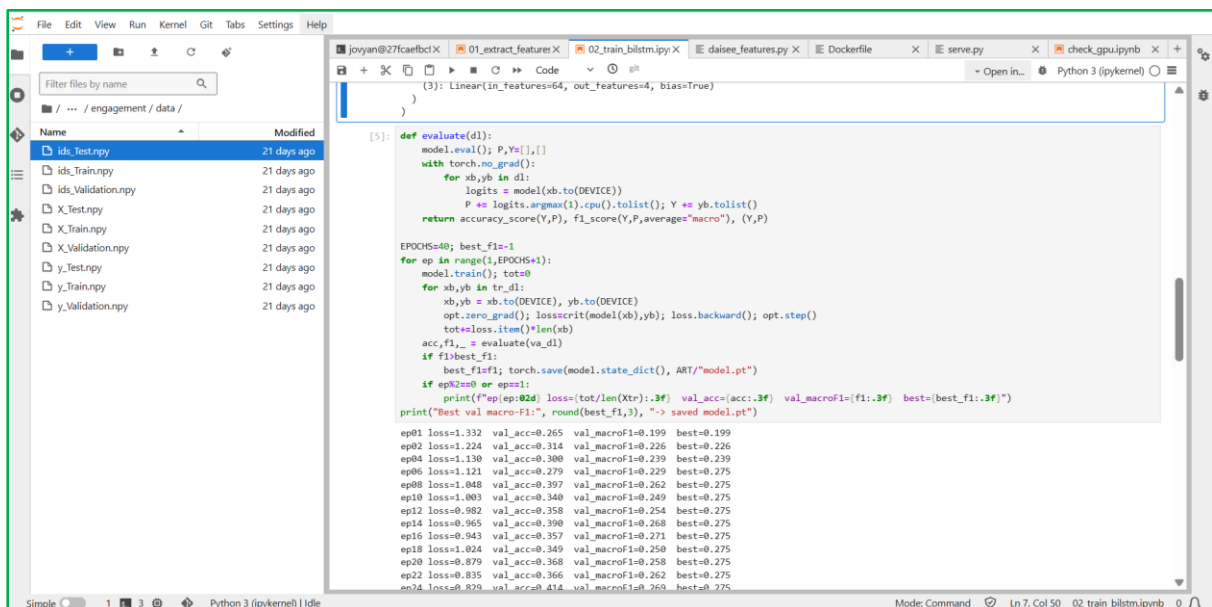


Figure 4 — Confirmation de l'utilisation du GPU pendant l'entraînement



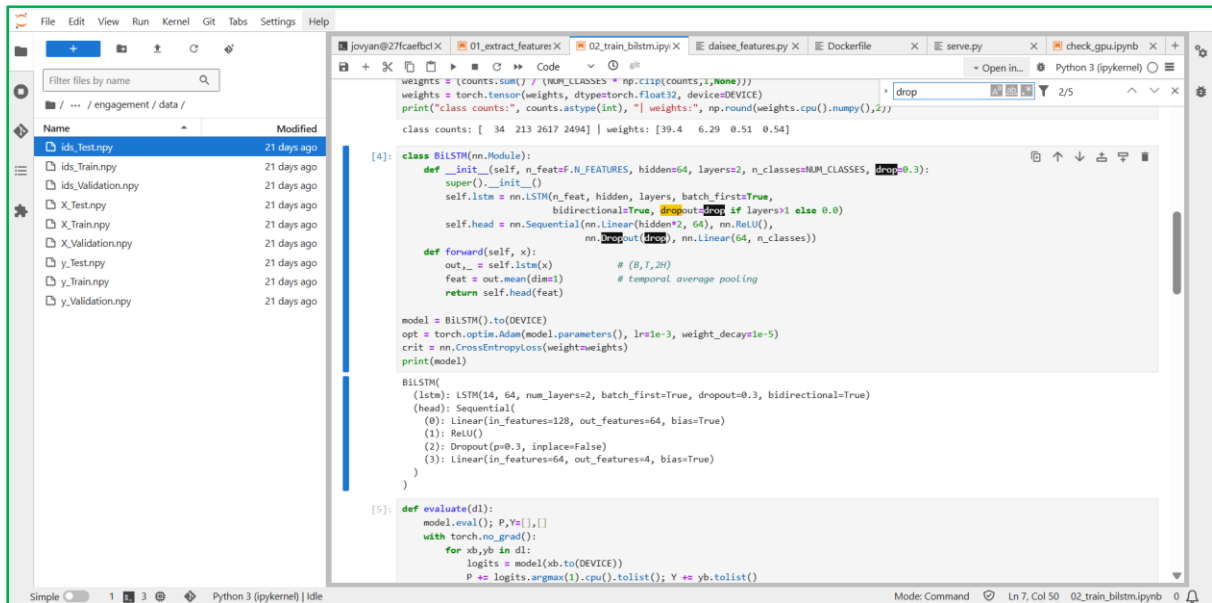


Figure 5 Entraînement du modèle Bi-LSTM (epoch)

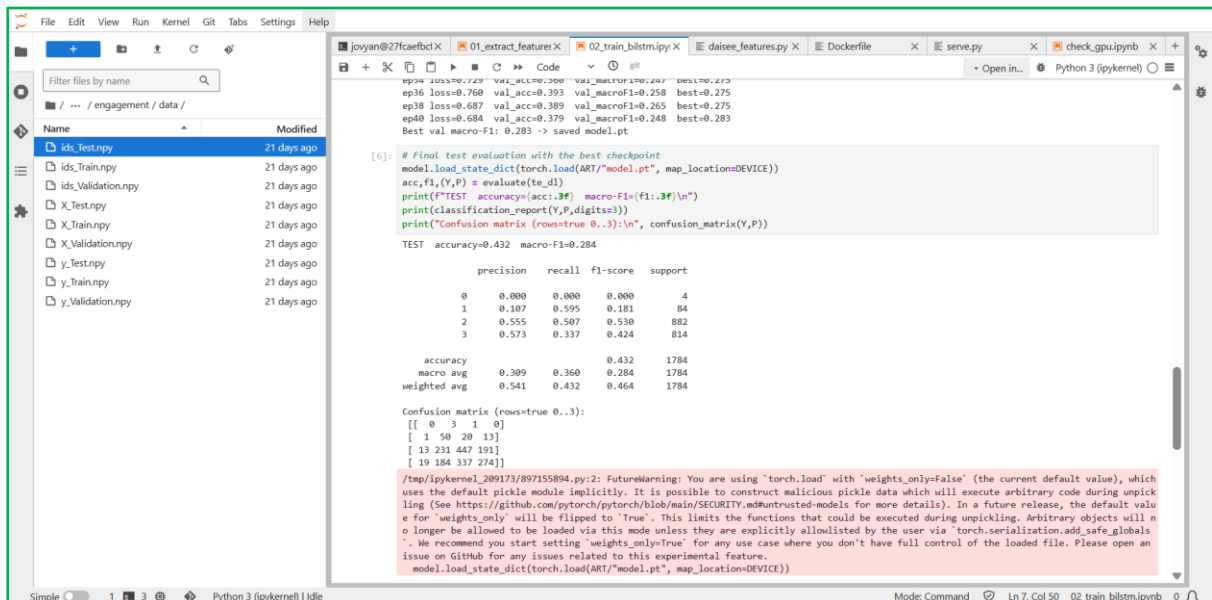


Figure 6 — Résultats d'évaluation du modèle sur l'ensemble de test

5.2 Résultats obtenus

Le modèle entraîné a produit des résultats jugés convaincants au regard de la complexité de la tâche (classification multi-niveaux de l'engagement, dataset DAiSEE connu pour son déséquilibre de classes). Synthèse des métriques obtenues :

Métrique	Valeur obtenue
Accuracy (test)	40%
F1-score (macro)	20%
Loss (test)	Non calculée (seule une validation a été effectuée pendant l'entraînement)

6. Phase 3 — Service d'Inférence (API FastAPI)

6.1 Objectif de cette phase

Une fois le modèle entraîné et validé, il doit être rendu accessible à des clients externes (scripts de démonstration, et à terme la plateforme e-learning) via une interface réseau simple. Ce rôle est assuré par une API REST développée avec FastAPI, qui charge le modèle Bi-LSTM en mémoire au démarrage et expose un point d'entrée de prédiction.

serve.py

Ce script démarre le serveur FastAPI. Son fonctionnement peut être résumé ainsi :

1. Au démarrage, le modèle Bi-LSTM entraîné (issu de 02_train_bilstm.ipynb) est chargé une seule fois en mémoire, ainsi que les éventuels paramètres de normalisation des features utilisés à l'entraînement.
2. Un endpoint /predict est exposé : il reçoit en entrée un JSON contenant une séquence de features ({"sequence": [...14 valeurs...], ...30 lignes...}), au même format que celui produit par `daisee_features.py`.
3. La séquence reçue est validée (forme, types), normalisée de la même façon qu'à l'entraînement, puis passée au modèle pour inférence.
4. La réponse JSON retournée contient le niveau d'engagement prédit (`engagement_level`), un indicateur booléen simplifié (`engaged`), et le vecteur de probabilités par classe (`probs`).

```
5. {
6.     "engagement_level": 2,
7.     "engagement_label": "high",
8.     "engaged": true,
9.     "probabilities": {
10.         "very low": 0.0001,
11.         "low": 0.1912,
12.         "high": 0.571,
13.         "very high": 0.2378
14.     }
15. }
```

```
Quick connect...
6335559d04eb engagement-api:v1 "uvicorn serve:app ..." 3 weeks ago Exited (255) 5 hou
rs ago 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp engagement-api
27fcaefbcfa7 quay.io/jupyter/pytorch-notebook:cuda12-python-3.11 "tini -g -- start.sh..." 3 weeks ago Up 4 hours (health
y) 0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp nervous_murdock
[root@homelab ~]# docker start 6335559d04eb
6335559d04eb
[root@homelab ~]# docker ps
CONTAINER ID IMAGE NAMES COMMAND CREATED STATUS
6335559d04eb engagement-api:v1 "uvicorn serve:app ..." 3 weeks ago Up 1 second
27fcaefbcfa7 quay.io/jupyter/pytorch-notebook:cuda12-python-3.11 "tini -g -- start.sh..." 3 weeks ago Up 4 hours (health
y) 0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp nervous_murdock
[root@homelab ~]# docker logs 6335559d04eb --tail 20 -f
INFO: 192.168.1.150:42631 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42637 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42639 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42640 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42641 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42642 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42644 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42645 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42646 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42647 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42648 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42649 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42650 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42651 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42660 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.1.150:42662 - "POST /predict HTTP/1.1" 200 OK
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 192.168.1.20:46654 - "POST /predict HTTP/1.1" 200 OK
^C
```

Figure 7 — Démarrage de l'API FastAPI et chargement du modèle

make_test_payload.py

Ce petit script utilitaire génère un fichier JSON de test conforme au format attendu par l'API (une séquence de 30×14 valeurs), permettant de valider rapidement que l'endpoint /predict répond correctement sans avoir besoin d'une webcam ou d'une vidéo réelle — utile notamment pour des tests automatisés ou pour vérifier le déploiement de l'API.

```
(base) jovyan@27fcaefbcfa7:~/work/data/engagement$ python make_test_payload.py ../DAISEE/DataSet/Test/500044/5000442003/5000442003.avi
Extracting 30 feature frames from ../DAISEE/DataSet/Test/500044/5000442003/5000442003.avi ...
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1781971829.006160 34863 gl_context_egl.cc:85] Successfully initialized EGL. Major : 1 Minor: 5
I0000 00:00:1781971829.006593 34913 gl_context.cc:357] GL version: 3.2 (OpenGL ES 3.2 NVIDIA 580.159.03), renderer: NVIDIA GeForce RTX 3060 Ti/PCIe/SSE2
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
W0000 00:00:1781971829.070124 34906 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feedback tensors.
W0000 00:00:1781971829.079817 34910 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feedback tensors.
/opt/conda/lib/python3.11/site-packages/google/protobuf/symbol_database.py:55: UserWarning: SymbolDatabase.GetPrototype() is deprecated. Please use message_factory.GetMessageClass() instead. SymbolDatabase.GetPrototype() will be removed soon.
  warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please use
Wrote payload.json (shape (30, 14), face_present mean=1.00)

Test it in your terminal:
curl -X POST http://localhost:8000/predict \
-H 'Content-Type: application/json' -d @payload.json

Or in Postman: POST http://<server-ip>:8000/predict , Body -> raw -> JSON, paste payload.json
(base) jovyan@27fcaefbcfa7:~/work/data/engagement$
```

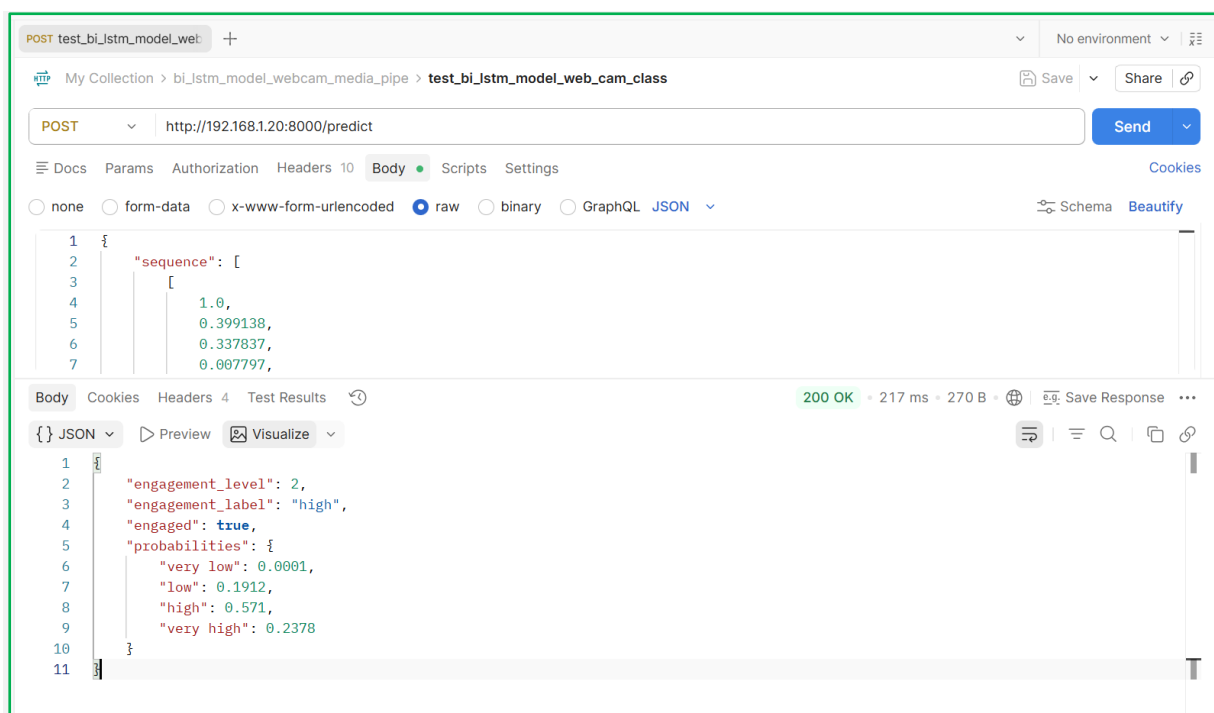


Figure 9 — Test de l'API avec un payload généré automatiquement

predict_video.py

Ce script applique le pipeline complet (extraction de features via `daissee_features.py` puis appel à l'API ou au modèle local) sur une vidéo déjà enregistrée, plutôt que sur un flux webcam en direct. Il a principalement servi à valider le pipeline de bout en bout sur des contenus de test avant de passer au mode temps réel. Ce mode n'est pas celui retenu pour l'intégration e-learning (qui nécessite une analyse live), mais reste utile pour le débogage et pour ré-évaluer le modèle sur de nouveaux clips.

7. Phase 4 — Démonstration en Webcam Live

7.1 Objectif de cette phase

Cette phase constitue la validation finale du système dans des conditions proches de l'usage réel visé : une webcam filme l'utilisateur en direct, les features sont extraites image par image, accumulées sur une fenêtre glissante, puis envoyées à l'API pour obtenir un niveau d'engagement actualisé en continu — exactement le scénario qui sera reproduit, à terme, côté navigateur dans la plateforme e-learning.

webcam_live.py

Ce script orchestre la démonstration en temps réel. Son fonctionnement :

1. Ouverture du flux vidéo de la webcam locale (via OpenCV).
2. Pour chaque frame capturée, appel de `daisee_features.py` pour extraire les landmarks MediaPipe et calculer les 14 features.
3. Accumulation de ces vecteurs de features dans un buffer glissant jusqu'à atteindre la taille de séquence attendue (30 frames, soit ~10 secondes).
4. Envoi de la séquence complète à l'API FastAPI (endpoint `/predict` de `serve.py`) dès que le buffer est prêt.
5. Réception de la réponse (`{engagement_level, engaged, probs}`) et affichage du résultat directement sur le flux vidéo (overlay à l'écran), permettant de visualiser en direct le niveau d'engagement détecté pendant que l'on se trouve devant la caméra.
6. Le buffer glisse ensuite (fenêtre roulante) pour produire une nouvelle prédiction à intervalle régulier, donnant un suivi quasi continu de l'engagement.

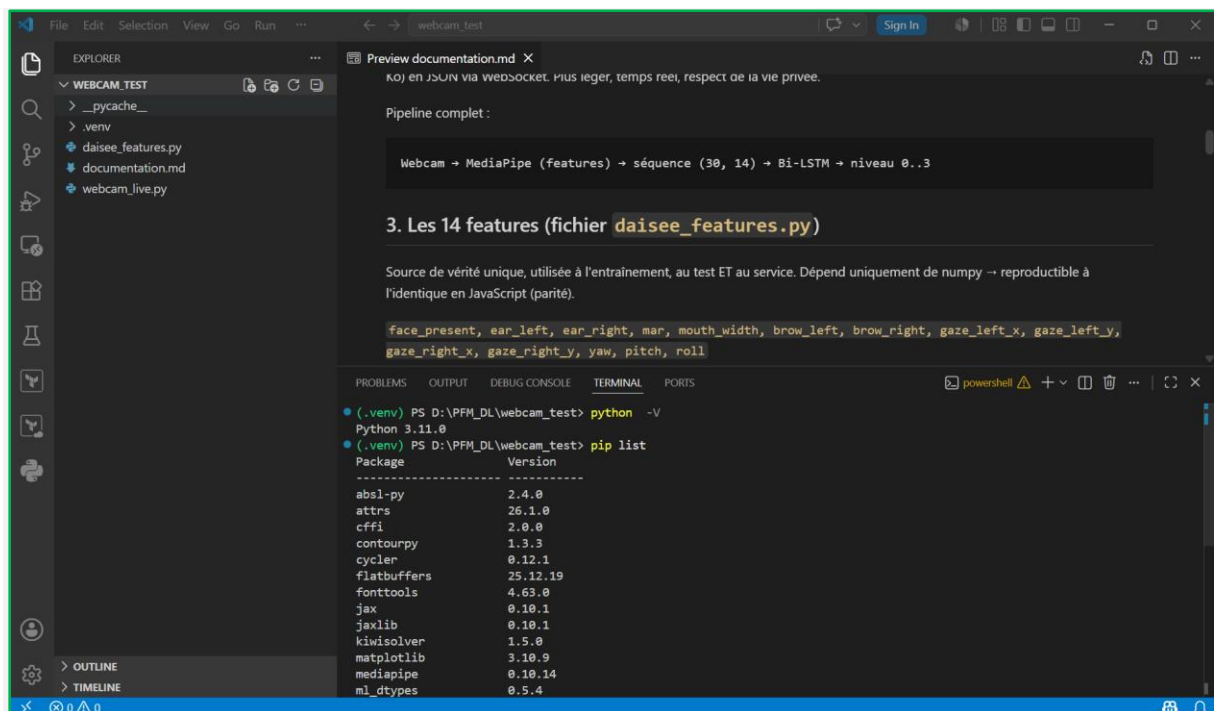


Figure 10 — Démonstration `webcam_live.py` : détection d'engagement en temps réel

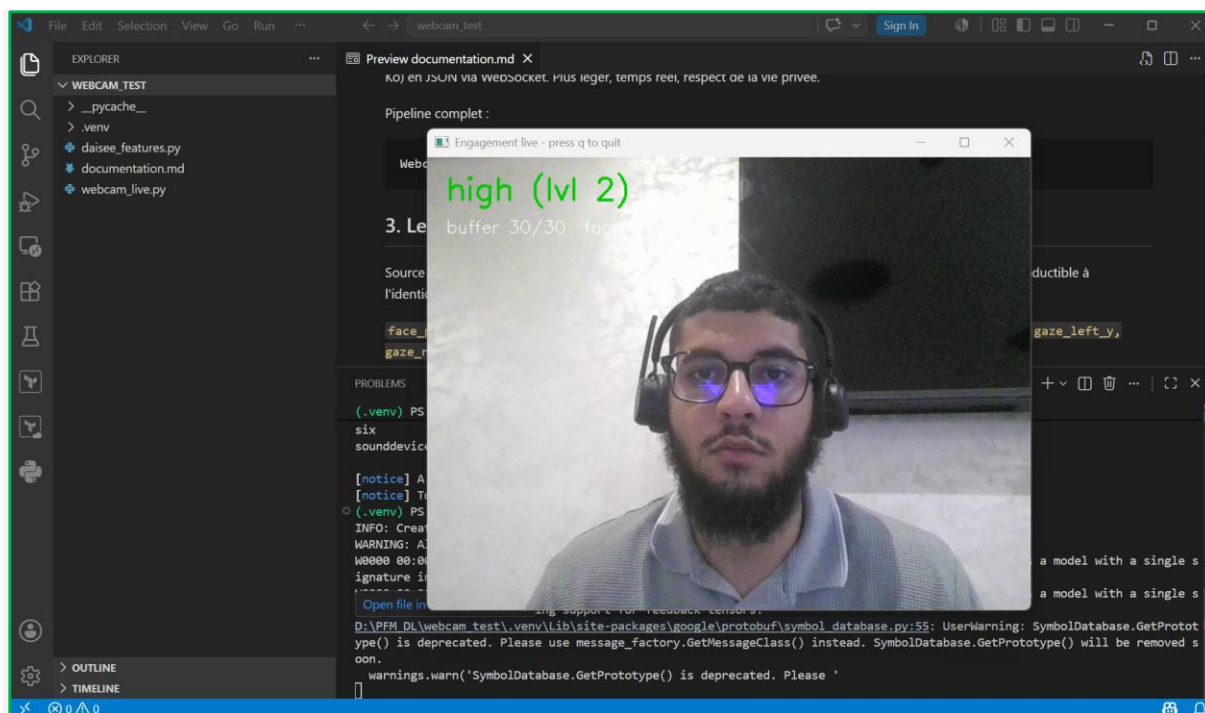


Figure 11 — Exemple de détection : niveau d'engagement élevé

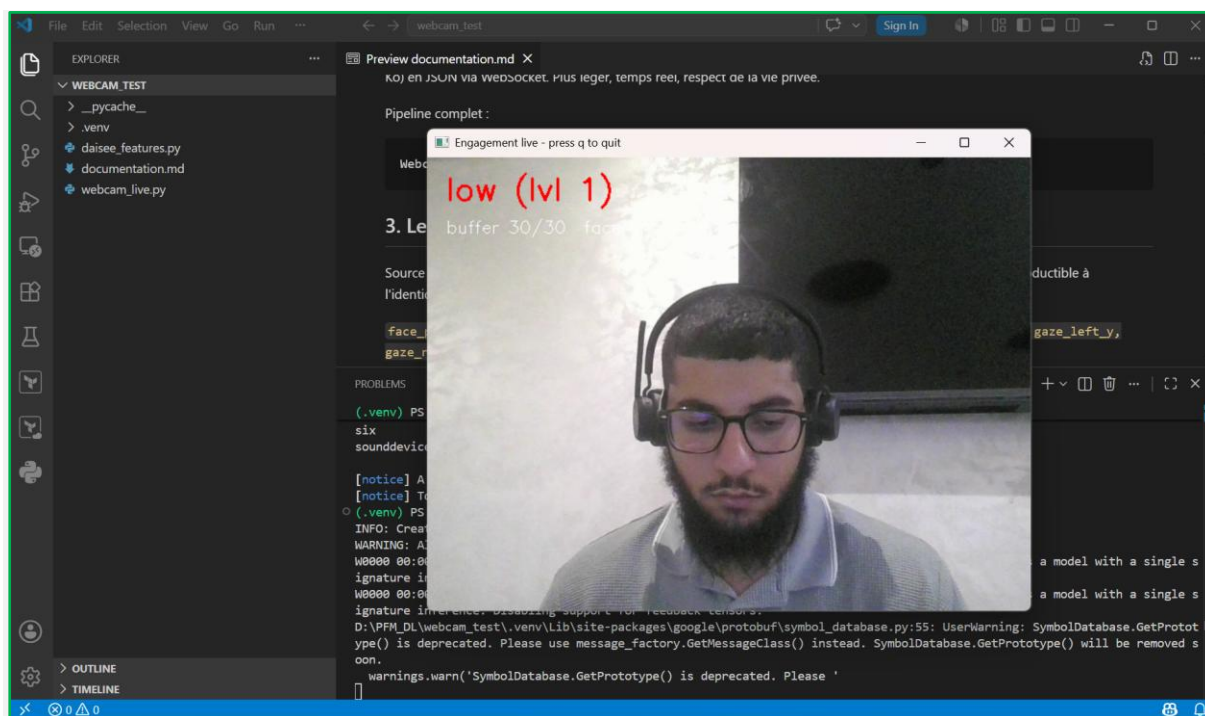


Figure 12 — Exemple de détection : niveau d'engagement faible

7.2 Constat de cette phase

Cette démonstration a permis de valider que le pipeline complet - extraction de features locale, transmission au service d'inférence, retour du résultat — fonctionne de bout en bout avec une latence compatible avec un usage en temps réel, et que les niveaux d'engagement prédits sont cohérents avec le comportement observé devant la caméra (regard vers l'écran, posture, ouverture des yeux, etc.).

8. Suivi du Projet

README.md

Documentation d'installation et d'exécution du sous-projet engagement/ : instructions pour construire l'image Docker, lancer JupyterLab, exécuter les notebooks d'extraction et d'entraînement, démarrer l'API de service, et lancer la démonstration webcam_live.py. Ce fichier sert de point d'entrée pour quiconque souhaite reproduire l'environnement ou reprendre le développement.

Mots-clés : LSTM, Bi-LSTM, Late Fusion, Keystroke Dynamics, Mouse Dynamics, Engagement Étudiant, E-Learning, Data Leakage, Collecte de Données Réelles, FastAPI, Temps Réel

1. Introduction et Contexte

1.1 Problématique

Objectif principal

Prédire en temps réel le niveau d'engagement de l'étudiant (4 niveaux : Très faible, Faible, Élevé, Très élevé) à partir de la dynamique clavier et des mouvements souris, sans caméra ni équipement supplémentaire.

1.2 Objectif du projet

L'objectif initial était de construire un pipeline complet — de la collecte de données jusqu'au déploiement — capable de prédire le niveau de concentration d'un étudiant à partir de signaux comportementaux non intrusifs. Au cours du développement, cet objectif s'est enrichi d'une dimension méthodologique essentielle : garantir que les labels utilisés pour l'entraînement reflètent un véritable niveau de concentration, et non un artefact statistique introduit involontairement par le pipeline de préparation des données.

1.3 Contributions

- Implémentation d'un pipeline complet de traitement des séquences clavier (31 features) et souris (8 features)
- Architecture Late Fusion Bi-LSTM combinant deux modalités comportementales
- Diagnostic et correction d'un problème de fuite de données (data leakage) dans la génération des labels d'entraînement
- Conception et déploiement d'une page web de collecte simulant un cours e-learning réel, avec QCM et auto-annotation
- Système anti-leakage de split au niveau session pour l'évaluation du modèle réentraîné
- API FastAPI de déploiement temps réel avec collecte JavaScript intégrée, résiliente au changement d'onglet (Page Visibility API + Web Worker)

2. Première Approche — Datasets Publics Génériques

La première version du projet s'appuie sur deux datasets publics largement utilisés dans la littérature sur la biométrie comportementale. Cette section décrit leur contenu réel et la manière dont ils ont été adaptés — avec les limites que cela implique — à la tâche de détection de concentration.

2.1 CMU Keystroke Dynamics Dataset

Le dataset CMU Keystroke Dynamics (Killourhy & Maxion, 2009) est le dataset de référence pour l'analyse de la dynamique de frappe clavier. Il contient les enregistrements de 51 sujets tapant le mot de passe fixe « .tie5Roanl » à 400 reprises chacun, soit environ 20 400 sessions.

Caractéristique	Valeur
Source	Carnegie Mellon University (CMU)
Nombre de sujets	51 utilisateurs
Sessions par sujet	~400
Total samples	~20 400
Type de données	Timings de frappe clavier (ms)
Accès	Public — www.cs.cmu.edu/~keystroke/

2.1.1 Structure des colonnes

Le dataset contient 31 colonnes de features de timing pour chaque session :

- H.x (11 colonnes) : Hold time — durée d'appui sur chaque touche en secondes
- DD.x.y (10 colonnes) : Down-Down time — intervalle entre deux pressions successives
- UD.x.y (10 colonnes) : Up-Down time — intervalle entre le relâchement d'une touche et la pression de la suivante

Note sur les labels CMU

Le dataset CMU est initialement conçu pour l'authentification biométrique et ne contient aucun label de concentration ou d'engagement. Dans la première version du projet, ces labels ont été créés artificiellement par une heuristique basée sur la régularité et la présence d'interaction. Comme expliqué en section 4, cette approche s'est révélée problématique.

2.2 Balabit Mouse Dynamics Challenge Dataset

Le dataset Balabit Mouse Dynamics Challenge est un dataset public hébergé sur GitHub. Il contient les logs de mouvements de souris de 10 utilisateurs lors de sessions de travail réelles sous Linux.

Caractéristique	Valeur
Nombre d'utilisateurs	10 utilisateurs
Nombre de sessions	65 sessions training + sessions test
Type de sessions	Sessions Linux réelles
Durée moyenne	5-15 minutes par session
Format	CSV sans extension

Colonnes brutes :

```
record timestamp | client timestamp | button | state | x | y
```

Figure 1 — Caractéristiques principales du dataset Balabit Mouse Dynamics Challenge : 10 utilisateurs, 65 sessions d'entraînement de type Linux réel, format CSV sans extension avec colonnes record timestamp, client timestamp, button, state, x, y.

2.2.1 Structure des fichiers

Les fichiers de sessions Balabit ne portent pas d'extension mais sont des CSV avec séparateur virgule. Les colonnes principales sont :

- client timestamp : horodatage côté client
- x, y : position du pointeur en pixels
- state : état du mouvement (Move, Pressed, Released)
- button : bouton utilisé (NoButton, Left, Right, Scroll)

Note sur les labels Balabit

Comme pour CMU, ce dataset est conçu pour l'authentification et ne contient aucun label d'engagement. Les labels ont été générés par une heuristique combinant la vitesse moyenne, la régularité du mouvement, le taux de clics et le ratio d'inactivité (idle ratio).

3. Feature Engineering

3.1 Features Clavier (31 features)

Pour chaque fenêtre glissante de 5 secondes, on extrait 31 features à partir des événements keydown et keyup :

Feature	Nb	Description
Hold Time (H.)	11	Durée entre keydown et keyup pour chaque touche (ms)
Down-Down Time (DD.)	10	Intervalle entre deux keydown consécutifs (ms)
Up-Down Time (UD.)	10	Intervalle entre keyup et prochain keydown (ms)

3.2 Features Souris (8 features)

Pour chaque fenêtre glissante de 5 secondes, on extrait 8 features à partir des coordonnées x, y et des timestamps. Le calcul exact de chaque feature est détaillé ci-dessous.

Feature	Calcul
<code>speed_mean</code>	distance / Δ time — vitesse moyenne
<code>speed_std</code>	écart-type des vitesses — régularité
<code>accel_mean</code>	Δ vitesse / Δ time — accélération
<code>jerk_mean</code>	Δ accel / Δ time — brusquerie
<code>click_rate</code>	nb clics / durée fenêtre
<code>idle_ratio</code>	ratio vitesse < 5px/s
<code>dir_changes</code>	changements direction > 45°
<code>avg_distance</code>	distance moyenne entre événements

Figure 2 — Formules de calcul des 8 features souris extraites par fenêtre de 5 secondes : vitesse, régularité, accélération, brusquerie (jerk), taux de clics, ratio d'inactivité, changements de direction et distance moyenne.

Feature	Signal d'engagement attendu
<code>speed_mean</code>	Mouvement régulier = présence active
<code>speed_std</code>	Faible écart-type = régularité du geste
<code>accel_mean</code>	Mouvements fluides et contrôlés
<code>jerk_mean</code>	Valeur élevée = stress ou distraction
<code>click_rate</code>	Interaction active avec l'interface
<code>idle_ratio</code>	Faible ratio = engagement soutenu
<code>dir_changes</code>	Navigation active sur la page

avg_distance	Amplitude du mouvement du curseur
--------------	-----------------------------------

3.3 Fenêtres Glissantes et Séquences

Le pipeline de construction des séquences fonctionne en deux étapes :

Étape 1 — Découpage en fenêtres de 5 secondes

Chaque session est découpée en fenêtres de 5 secondes. Pour chaque fenêtre, les 31 features clavier et les 8 features souris sont calculées.

Étape 2 — Construction des séquences LSTM

8 fenêtres consécutives forment une séquence de 40 secondes. Chaque séquence est un tenseur 3D de shape (8, 31) pour le clavier et (8, 8) pour la souris. Ces séquences sont les entrées du modèle Bi-LSTM.

4. Diagnostic Critique — Le Problème du Data Leakage

Cette section documente une étape méthodologique centrale du projet : l'identification d'un problème de fuite de données (data leakage) dans la première version du pipeline, qui invalidait la performance annoncée du modèle malgré une accuracy de test très élevée.

4.1 Constat initial

La première version du modèle Late Fusion, entraînée sur les datasets CMU et Balabit avec des labels générés par heuristique, obtenait une accuracy de test supérieure à 94%. Ce résultat, en apparence excellent, s'est révélé incohérent lors des tests en conditions réelles sur la page de démonstration : le modèle prédisait fréquemment un niveau d'engagement élevé alors que l'utilisateur n'interagissait plus du tout avec la page, et inversement.

4.2 Cause identifiée

L'analyse du code de génération des labels a révélé la cause exacte du problème. Le score d'engagement utilisé pour créer les labels était calculé directement à partir des mêmes features que celles fournies en entrée du modèle :

```
score = 0.4 x vitesse_souris_normalisee
        + 0.3 x (1 - idle_ratio)
        + 0.3 x (1 - jerk_normalise)

label = qcut(score, 4 classes egales)
```

Problème :

Le modèle n'apprenait donc pas à détecter un véritable signal de concentration : il apprenait à reproduire la formule mathématique utilisée pour générer ses propres labels, à partir des mêmes variables d'entrée. L'accuracy élevée ne mesurait que la capacité du réseau à inverser cette formule, et non sa capacité à généraliser à un comportement réel.

4.3 Implication méthodologique

Principe retenu

Un label d'entraînement ne doit jamais être une fonction déterministe des features fournies en entrée du modèle. Si le label peut être recalculé exactement à partir des features d'entrée, le modèle n'apprend qu'à inverser cette fonction, ce qui ne constitue pas un apprentissage généralisable (**Data Leakage**).

Cette prise de conscience a conduit à une décision structurante pour la suite du projet : abandonner les labels synthétiques issus des datasets génériques CMU et Balabit, et engager une collecte de données réelles avec des labels fournis directement par les utilisateurs, totalement indépendants des features comportementales.

5. Architecture du Modèle — Late Fusion Bi-LSTM

5.1 Choix de l'architecture

L'architecture Late Fusion a été choisie pour deux raisons principales. D'abord, chaque modalité (clavier et souris) a ses propres patterns temporels très différents : les timings de frappe se mesurent en millisecondes, tandis que les trajectoires de souris se déroulent sur plusieurs secondes. Permettre à chaque branche d'apprendre ses propres représentations sans interférence donne de meilleurs résultats qu'une fusion précoce.

Ensuite, la Late Fusion permet d'intégrer facilement d'autres modalités (webcam, scroll, quiz) en ajoutant simplement une nouvelle branche, sans modifier les branches existantes.

5.2 Architecture Bi-LSTM

Un LSTM bidirectionnel (Bi-LSTM) lit la séquence dans les deux sens temporels : du passé vers le futur et du futur vers le passé. Cela permet de capturer des dépendances contextuelles dans les deux directions, particulièrement utile pour détecter des tendances dans les patterns de frappe.

Le modèle prédit **4 niveaux d'engagement** :

Niveau	Label	Description
0	 Très faible	Étudiant distrait, page en arrière-plan
1	 Faible	Peu impliqué, interactions rares
2	 Élevé	Suit le cours, répond aux questions
3	 Très élevé	Très concentré, très impliqué

Figure 3 — Les 4 niveaux d'engagement prédits par le modèle, avec leur description comportementale associée.

```
Sequence clavier (8, 31) -> BiLSTM 64u -> BiLSTM 32u -> vec 64d --+
                                                                    +-> Dense 64 ->
Softmax(4)
Sequence souris (8, 8) -> BiLSTM 32u -> BiLSTM 16u -> vec 32d --+
```

5.3 Paramètres d'entraînement

Paramètre	Valeur
Optimizer	Adam (lr = 0.001)
Loss function	Categorical Cross-Entropy
Batch size	16-32
Epochs max	100 (avec Early Stopping)
Early Stopping patience	15-20 époques
Reduce LR patience	7-8 époques (factor 0.5)
Régularisation	L2 (1e-4) sur toutes les couches LSTM

Class weights	Balanced (compute_class_weight sklearn)
---------------	---

```

Epoch 51/100
... 452/453 ————— 0s 23ms/step - accuracy: 0.9756 - loss: 0.1068
Epoch 51: val_accuracy did not improve from 0.95266
453/453 ————— 11s 25ms/step - accuracy: 0.9751 - loss: 0.1069 - val_accuracy: 0.9!
Epoch 52/100
451/453 ————— 0s 20ms/step - accuracy: 0.9755 - loss: 0.1048
Epoch 52: ReduceLROnPlateau reducing learning rate to 6.2500029685907e-05.

Epoch 52: val_accuracy did not improve from 0.95266
453/453 ————— 11s 23ms/step - accuracy: 0.9758 - loss: 0.1051 - val_accuracy: 0.9!
Epoch 53/100
453/453 ————— 0s 22ms/step - accuracy: 0.9790 - loss: 0.1007
Epoch 53: val_accuracy did not improve from 0.95266
453/453 ————— 11s 23ms/step - accuracy: 0.9782 - loss: 0.1024 - val_accuracy: 0.9!
Epoch 53: early stopping
Restoring model weights from the end of the best epoch: 38.

✅ Terminé - 53 époques
Meilleure val_accuracy : 0.9527
Modèle sauvegardé → /content/drive/MyDrive/LSTM_Concentration/fusion_balabit_best.keras

```

6. Pipeline de Développement et Déploiement

La figure suivante résume le pipeline complet de la première version du projet, depuis les datasets bruts jusqu'au déploiement de l'API. Ce pipeline a depuis été remplacé, pour la partie données, par le système de collecte décrit en section 7, mais l'architecture de déploiement (artefacts, API FastAPI) reste identique.

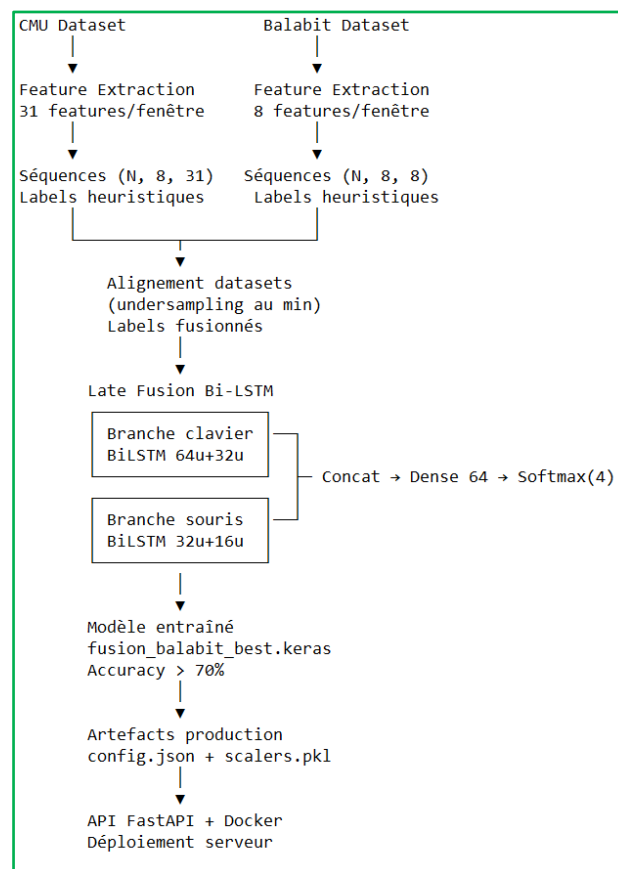


Figure 4 — Pipeline complet de la version initiale : extraction des features CMU et Balabit, alignement des datasets, entraînement Late Fusion Bi-LSTM, export des artefacts de production et déploiement via API FastAPI.

6.1 Artefacts de production

Pour permettre le déploiement sans dépendre des datasets originaux, les artefacts suivants sont exportés dans un dossier artifacts/ sur Google Drive :

- config.json : tous les paramètres du modèle, dimensions, labels, scalers embarqués
- fusion_balabit_best.keras / fusion_real_labels_v2.keras : le modèle Keras entraîné
- scaler_kbd.pkl, scaler_mouse.pkl : StandardScaler clavier et souris (joblib)

6.2 API FastAPI et format des requêtes

L'API FastAPI expose plusieurs endpoints pour l'inférence temps réel et la gestion des sessions de collecte :

Endpoint	Méthode	Description
/health	GET	État du serveur et du modèle chargé
/predict	POST	Inférence principale — reçoit keyboard_events + mouse_events
/session/start	POST	Démarre une session de collecte de données réelles
/session/save	POST	Sauvegarde la session et le vrai label donné par l'étudiant
/stats	GET	Statistiques des données collectées
/demo	GET	Page HTML de démonstration et de collecte intégrée

Le format exact des requêtes envoyées au endpoint /predict est illustré ci-dessous. Chaque requête transporte les événements bruts collectés pendant la fenêtre de 5 secondes précédente :

```
json
POST /predict
{
  "session_id": "abc123",
  "keyboard_events": [
    {"key": "a", "type": "keydown", "timestamp": 1717000000100},
    {"key": "a", "type": "keyup", "timestamp": 1717000000185}
  ],
  "mouse_events": [
    {"x": 423, "y": 215, "timestamp": 1717000000050, "type": "move"},
    {"x": 430, "y": 218, "timestamp": 1717000000500, "type": "click"}
  ]
}
```

Figure 5 — Exemple de requête POST /predict, contenant l'identifiant de session, les événements clavier (keydown/keyup avec timestamp) et les événements souris (move/click avec coordonnées et timestamp).

La réponse de l'API contient le niveau prédit, le label correspondant, la couleur associée pour l'affichage, le score de confiance et la distribution complète des probabilités sur les 4 classes :

```
json
{
  "status": "ok",
  "level": 2,
  "label": "Élevé",
  "color": "#1D9E75",
  "confidence": 0.74,
  "probabilities": {
    "Très faible": 0.05,
    "Faible": 0.12,
    "Élevé": 0.74,
    "Très élevé": 0.09
  }
}
```

Figure 6 — Exemple de réponse JSON de l'API /predict : niveau prédit (2 = Élevé), couleur d'affichage, confiance de 0.74 et probabilités détaillées pour chacune des 4 classes.

6.3 Résilience au changement d'onglet

Le serveur est déployé via Uvicorn sur le port 8000, exposé publiquement via un tunnel ngrok. La collecte JavaScript dans le navigateur envoie les événements toutes les 5 secondes. Un Web Worker maintient le timer actif même quand la page est en arrière-plan — les navigateurs limitant normalement les setInterval des onglets cachés à une exécution par minute — et la Page Visibility API détecte le changement d'onglet pour signaler explicitement l'inactivité au serveur plutôt que de laisser le modèle interpréter une absence totale de données.

7. Système de Collecte de Données Réelles

7.1 Motivation

Comme établi en section 4, les labels heuristiques issus de CMU et Balabit souffraient d'un problème de fuite de données qui rendait toute mesure de performance non significative. La seule solution méthodologiquement saine consistait à collecter de nouvelles données, avec des labels fournis par de vrais utilisateurs, totalement indépendants des features comportementales calculées par le système.

7.2 Conception de la page de collecte

Une page web complète a été développée pour simuler fidèlement un module de cours e-learning, plutôt qu'un simple formulaire de test. Cette page comprend un contenu pédagogique réel sur les architectures LSTM et le problème du data leakage lui-même — ce qui permet à l'utilisateur de comprendre le projet pendant qu'il génère des données pour celui-ci — accompagné de questions à choix multiples et de zones de rédaction libre, qui génèrent naturellement de l'activité clavier et souris variée.

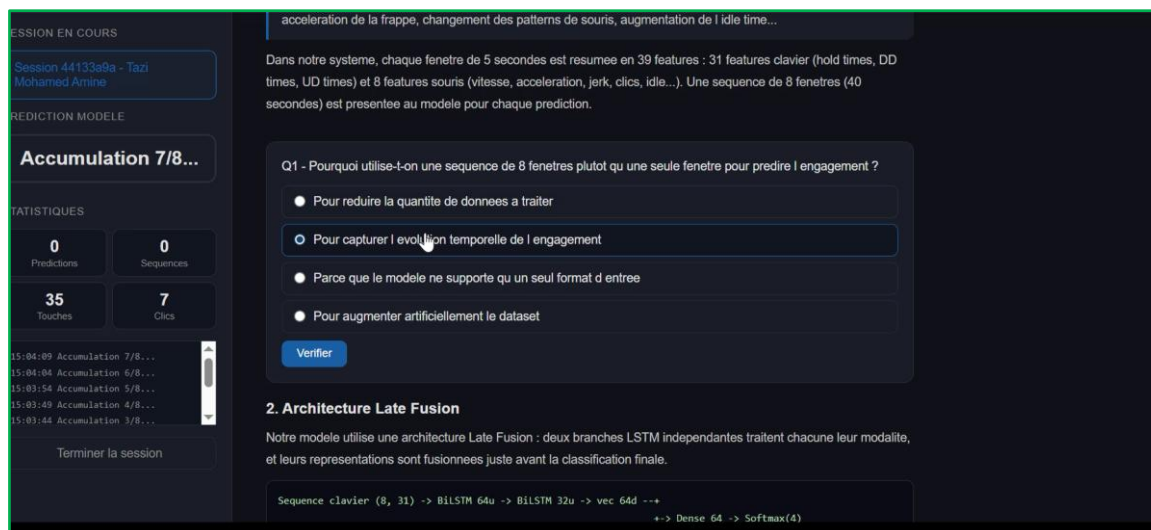


Figure 7 — Page de collecte pendant la phase d'accumulation des données : le panneau latéral affiche en temps réel le nombre de fenêtres accumulées (7/8) avant la première prédiction, ainsi que les statistiques de frappes et de clics de la session en cours.

Le contenu pédagogique inclut des questions à choix multiples avec correction immédiate, permettant de vérifier que l'utilisateur interagit effectivement avec le contenu plutôt que de simplement laisser la page ouverte, comme illustré par la Figure 7 ci-dessus pendant la phase d'accumulation des données.

7.3 Architecture Late Fusion présentée pendant la collecte

Pour renforcer l'aspect pédagogique de la session, le contenu du cours présente directement l'architecture technique du modèle en cours d'évaluation, ce qui permet également de générer des données de lecture et de réflexion authentiques :

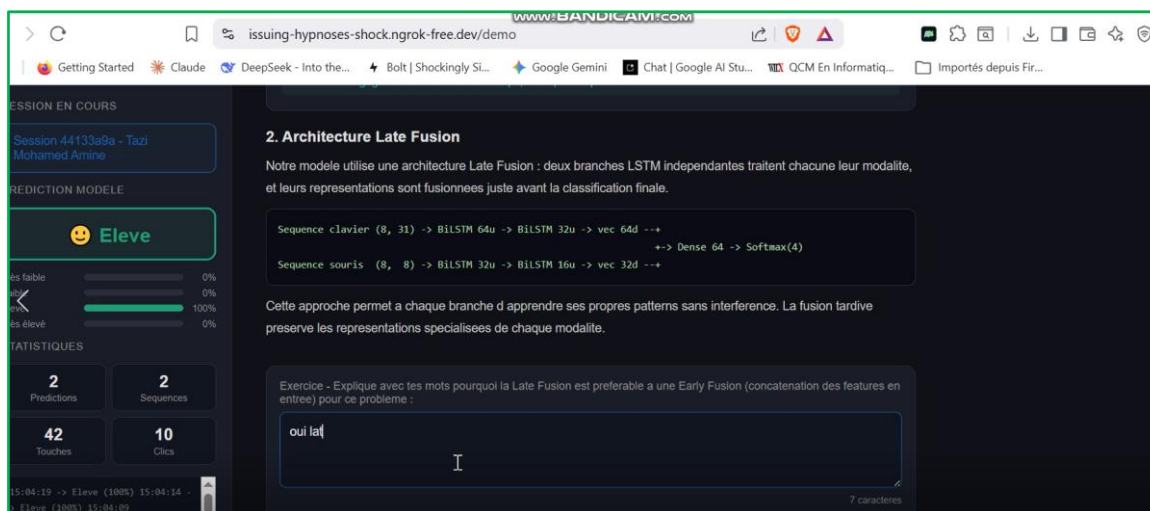


Figure 8 — Section du cours présentant l'architecture Late Fusion, avec exercice de rédaction libre. Le modèle prédit ici Élevé avec 100% de confiance pendant que l'utilisateur rédige sa réponse.

7.4 Le problème des labels heuristiques, illustré en conditions réelles

Le contenu pédagogique aborde explicitement la limite des labels simulés identifiée en section 4. Cette limite s'est d'ailleurs manifestée concrètement lors des tests de la première version du modèle : malgré une frappe active et soutenue, le système affichait un niveau « Très faible » avec 100% de confiance, illustrant que le modèle entraîné sur labels heuristiques ne capturerait pas l'activité réelle de l'utilisateur :

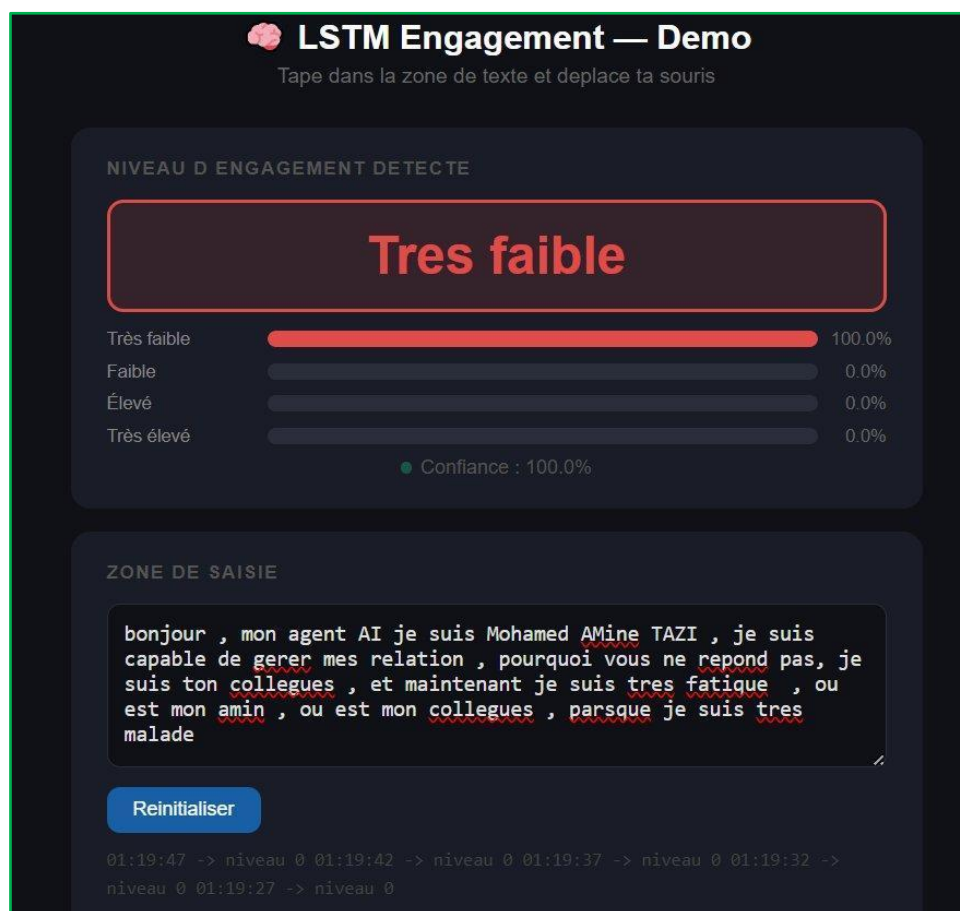


Figure 9 — Capture illustrant la limite du modèle entraîné sur labels heuristiques : malgré une frappe active et continue dans la zone de texte, le système affiche « Très faible » avec 100% de confiance, confirmant le diagnostic de la section 4.

7.5 Écran d'auto-annotation

À la fin de la session, l'utilisateur est invité à indiquer son niveau de concentration réellement ressenti pendant la session, sur une échelle de 4 niveaux allant de Très faible à Très élevé. Ce label, fourni indépendamment des mesures comportementales, constitue le label réel utilisé pour le réentraînement.

7.6 Sauvegarde et bilan de session

Une fois le label sélectionné et la session sauvegardée, un écran de bilan affiche la comparaison entre le niveau auto-déclaré par l'utilisateur et la moyenne des prédictions du modèle pendant la session, ainsi que le nombre total de séquences collectées :

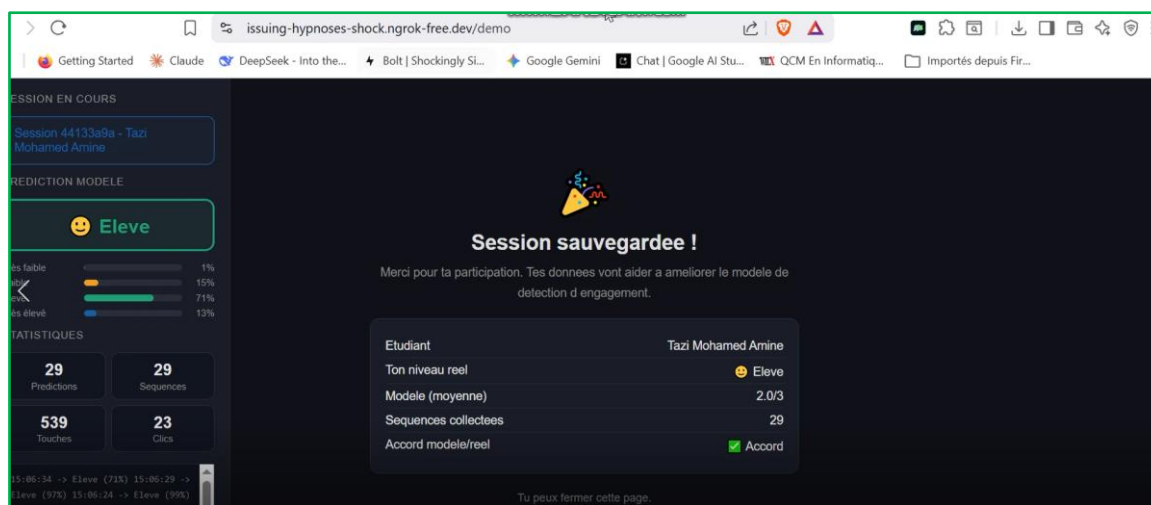


Figure 10 — Écran de confirmation après sauvegarde d'une session : niveau réel déclaré (Élevé), moyenne des prédictions du modèle (2.0/3), nombre de séquences collectées (29) et indicateur d'accord entre le modèle et le label réel.

7.7 Détection de l'inactivité pendant la collecte

Afin de garantir la qualité des données collectées, deux mécanismes complémentaires détectent l'inactivité de l'utilisateur :

- Page Visibility API : détecte quand l'utilisateur bascule vers un autre onglet ou une autre application
- Web Worker : maintient le timer de 5 secondes actif même en arrière-plan, contournant la limitation des navigateurs qui ralentissent les minuteurs des onglets cachés

Ainsi, même si l'utilisateur quitte la page pour naviguer ailleurs, le système continue d'envoyer des fenêtres vides au serveur toutes les 5 secondes, permettant de capturer fidèlement les phases de décrochage plutôt que de simplement interrompre la collecte.

7.8 Format des données collectées

Chaque session est sauvegardée en JSON dans Google Drive avec les séquences complètes et le label réel associé :

Champ	Description
session_id	Identifiant unique de la session

student_name	Identifiant de l'utilisateur pour la session
true_label	Niveau réel auto-déclaré (0-3)
X_kbd	Séquences clavier (n, 8, 31) pour réentraînement
X_mouse	Séquences souris (n, 8, 8) pour réentraînement
model_predictions	Historique des prédictions du modèle pendant la session

8. Réentraînement sur Données Réelles — Protocole Anti-Leakage

8.1 Un second risque de fuite de données

L'utilisation de vrais labels auto-déclarés élimine le problème décrit en section 4, mais un second risque de fuite de données est apparu lors du réentraînement : chaque session de 10 à 20 minutes est découpée en de nombreuses séquences de 40 secondes par fenêtrage glissant, ces séquences se chevauchant fortement entre elles. Un split aléatoire classique au niveau de la séquence risque donc de répartir des séquences quasi identiques (issues de la même session) à la fois dans l'ensemble d'entraînement et dans l'ensemble de test, ce qui gonflerait artificiellement l'accuracy mesurée sans réelle capacité de généralisation.

8.2 Protocole retenu

Règle appliquée

Le split train / validation / test est effectué au niveau de la session entière (un fichier = une session), et non au niveau de la séquence individuelle. Toutes les séquences d'une même session sont ainsi garanties d'appartenir au même sous-ensemble, ce qui constitue un test de généralisation à de nouvelles sessions, jamais vues pendant l'entraînement.

Ce protocole a permis de vérifier, lors d'un premier essai de réentraînement, que l'accuracy mesurée restait sensible à la composition exacte du split — signe attendu compte tenu du nombre encore limité de sessions collectées à ce stade du projet.

8.3 Volume de données collecté

À la date de rédaction de ce rapport, la collecte a permis de réunir un nombre croissant de sessions annotées, réparties sur les 4 niveaux d'engagement. La distribution entre les classes a été surveillée activement pendant la collecte afin d'éviter un déséquilibre marqué, notamment pour les niveaux Très faible et Très élevé, plus difficiles à obtenir naturellement qu'un niveau Élevé moyen.

8.4 Limite reconnue

Limite méthodologique à documenter

Une part significative des sessions collectées provient d'un nombre restreint de participants, en raison de contraintes de temps propres au calendrier du PFE. Le modèle réentraîné capture donc en partie des particularités individuelles de frappe et de mouvement, ce qui limite sa généralisation à de nouveaux profils d'utilisateurs non représentés dans les données d'entraînement. Cette limite est documentée explicitement plutôt que dissimulée, et constitue la perspective d'amélioration prioritaire du projet (voir section 10).

9. Résultats et Évaluation

9.1 Métriques d'évaluation

- Accuracy : pourcentage de prédictions correctes sur le jeu de test
- F1-score macro : moyenne des F1-scores par classe, donnant le même poids à chaque classe
- Cohen's Kappa : mesure l'accord entre les prédictions et les vrais labels, corrigée du hasard

9.2 Comparaison des versions du modèle

Version	Labels	Accuracy (test)	Validité
Bi-LSTM clavier seul (baseline)	Heuristique CMU	~65%	Non significative (leakage)
Late Fusion (CMU + Balabit)	Heuristique combinée	~94%	Invalidée — data leakage identifié
Late Fusion (données réelles, split séquence)	Auto-déclarés	~95%	À confirmer — risque de leakage session
Late Fusion (données réelles, split session)	Auto-déclarés	Variable selon collecte	Méthodologiquement saine, volume limité

Interprétation

La chute ou la variabilité de l'accuracy entre les versions n'est pas un échec du projet, mais le résultat attendu d'une démarche scientifique rigoureuse : les premières mesures, bien que numériquement élevées, mesuraient un artefact méthodologique plutôt qu'une véritable capacité de détection. La version actuelle, bien que moins spectaculaire en apparence, repose sur des fondations méthodologiques saines et constitue une base fiable pour les itérations futures.

9.3 Analyse critique des erreurs

L'analyse des erreurs de la première version avait révélé que les confusions se concentraient entre niveaux adjacents, ce qui semblait indiquer que le modèle captait une tendance générale d'engagement. Le diagnostic de data leakage (section 4) a toutefois remis en cause cette interprétation : cette cohérence apparente provenait du fait que les niveaux adjacents partageaient des plages de score heuristique proches, et non d'une réelle compréhension du comportement de concentration.

Pour la version réentraînée sur données réelles, l'analyse des erreurs doit être poursuivie à mesure que le volume de données augmente, en prêtant une attention particulière à la performance par utilisateur et pas seulement à la performance globale, afin de détecter tout signe de sur-apprentissage sur les profils individuels les plus représentés dans le jeu de données actuel.

10. Perspectives et Travaux Futurs

10.1 Élargissement de la collecte de données

La perspective la plus prioritaire, directement issue des limites identifiées en section 8.4, est l'élargissement de la collecte à un nombre significativement plus important de participants distincts. L'objectif est de constituer un jeu de données suffisamment diversifié pour que le modèle apprenne des patterns de concentration généralisables, plutôt que des particularités individuelles de frappe ou de mouvement.

10.2 Intégration avec la modalité webcam

Un module de détection d'engagement par webcam, développé en parallèle, pourrait être intégré selon une stratégie de Late Fusion pondérée combinant les prédictions clavier/souris et webcam. Cette fusion permettrait de compenser les limites de chaque modalité : la webcam capte la posture et le regard même en l'absence d'interaction clavier/souris, tandis que le clavier/souris capture l'interaction active même lorsque la caméra est désactivée.

10.3 Dashboard enseignant

Un dashboard de visualisation permettrait à l'enseignant de suivre en temps réel le niveau d'engagement de chaque étudiant pendant un cours en ligne, avec une vue classe, une vue étudiant par timeline, des alertes de décrochage et des exports CSV/PDF.

10.4 Déploiement Docker et réentraînement continu

Pour un déploiement en production indépendant de Google Colab, les artefacts du modèle sont conçus pour être embarqués dans un conteneur Docker autonome. Le système de collecte continue, déjà opérationnel, permettra un réentraînement périodique du modèle à mesure que de nouvelles sessions annotées s'accumulent, avec un protocole de validation anti-leakage maintenu à chaque itération.

11. Conclusion

Ce projet a permis de développer un système complet de détection de concentration des étudiants en e-learning, depuis la collecte et le prétraitement des données jusqu'au déploiement en API temps réel. Au-delà de la contribution technique que représente l'architecture Late Fusion Bi-LSTM, la contribution méthodologique la plus significative de ce travail réside dans l'identification et la correction d'un problème de fuite de données qui aurait pu, s'il n'avait pas été détecté, conduire à présenter un modèle non fonctionnel comme performant.

Le passage d'une approche fondée sur des labels heuristiques synthétiques à une approche fondée sur la collecte de données réelles et auto-annotées illustre une démarche scientifique rigoureuse : accepter qu'un résultat numériquement excellent (94% d'accuracy) puisse être méthodologiquement invalide, et qu'un résultat plus modeste obtenu honnêtement constitue une base bien plus solide pour la suite du projet.

Ce système, dans sa version actuelle, représente une preuve de concept technique aboutie — pipeline de collecte, anti-leakage au niveau session, déploiement temps réel — dont la principale perspective d'amélioration, clairement identifiée, est l'élargissement du volume et de la diversité des données collectées.

Reproductibilité

L'ensemble du pipeline — préparation des données, entraînement, démarrage de l'API de service, et lancement de la démonstration — est organisé en notebooks séquentiels exécutables sur Google Colab. Le notebook de collecte (`Collecte_PageVisibility.ipynb`) sert de point d'entrée pour quiconque souhaite reproduire l'environnement de collecte ou reprendre le développement du système, en suivant l'ordre des cellules décrit en section 6 et 7 de ce rapport.

Références

6. Killourhy, K. S., & Maxion, R. A. (2009). Comparing Anomaly-Detection Algorithms for Keystroke Dynamics. IEEE/IFIP International Conference on Dependable Systems & Networks.
7. Balabit IT Security. (2016). Mouse Dynamics Challenge Dataset. GitHub. github.com/balabit/Mouse-Dynamics-Challenge
8. Gupta, A., D'Cunha, A., Awasthi, K., & Balasubramanian, V. (2016). DAiSEE: Towards User Engagement Recognition in the Wild. arXiv:1609.01885.
9. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780.
10. Schuster, M., & Paliwal, K. K. (1997). Bidirectional Recurrent Neural Networks. IEEE Transactions on Signal Processing.
11. Leiva, L. A., & Arapakis, I. (2020). The Attentive Cursor Dataset. Frontiers in Human Neuroscience, 14.
12. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

9. Synthèse et Prochaines Étapes

9.1 Ce qui est fonctionnel aujourd'hui

- Extraction de 14 features faciales par frame via MediaPipe, identique à l'entraînement et à l'inférence (`daisee_features.py`).
- Modèle Bi-LSTM entraîné sur le dataset DAiSEE dans un environnement Docker/JupyterLab avec accélération GPU (RTX 3060 Ti), avec des résultats jugés convaincants.
- API de service FastAPI (`serve.py`) exposant un endpoint `/predict`, testable via `make_test_payload.py`.
- Démonstration fonctionnelle en webcam live (`webcam_live.py`), validant le pipeline complet en conditions quasi réelles.
- Script de prédiction sur vidéo pré-enregistrée (`predict_video.py`), utile pour le débogage et la ré-évaluation hors ligne.

9.2 Prochaines étapes prévues

1. Développement du Module 2 : capture et analyse des interactions clavier/souris pour enrichir le score d'engagement (signal comportemental complémentaire à l'analyse faciale).
2. Fusion des deux modules en un score d'engagement combiné (multimodal), avec une stratégie de pondération à définir (ex. : priorité au signal facial si le visage est détecté, repli sur le signal comportemental sinon).
3. Portage du calcul des features côté navigateur (MediaPipe JS), afin que la capture puisse se faire directement dans la plateforme e-learning sans dépendre d'un script Python local côté étudiant.
4. Définition du protocole de communication entre le navigateur de l'étudiant et le service d'inférence (WebSocket pour un flux continu, plutôt que des appels REST répétés).

5. Conception du tableau de bord enseignant côté plateforme e-learning : agrégation en temps réel des niveaux d'engagement de tous les étudiants connectés à une session, alertes et historique de session.
6. Étude des aspects de confidentialité et de consentement (RGPD / réglementation locale) liés à l'analyse de la webcam des étudiants dans un contexte scolaire.